

Python Unit1 – (4 marks)

List Any Four Features of Python

1. **Simple and Easy to Learn**

Python has very simple and clear syntax. It looks like English statements, so beginners can learn it easily.

2. **High Level Programming Language**

Python is a high level language. Programmers do not need to worry about memory management and low-level tasks.

3. **Platform Independent**

Python programs can run on Windows, Linux, and Mac operating systems. We do not need to change the code for different platforms.

4. **Dynamically Typed Language**

In Python, we do not need to declare the data type of a variable. The type is automatically decided when we assign a value.

5. **Extensive Library Support**

Python provides a large number of built-in libraries. These libraries help programmers complete tasks easily and quickly.

Explain the Concept of Mutability

1. Mutability means whether we can change the content of an object after it is created. If the object cannot be changed, it is called immutable.
 2. In immutable data types, if we try to modify the value, a new object is created. Examples of immutable types are int, float, bool, and str.
 3. Mutable objects can be changed after they are created. The changes will affect the same object in memory.
 4. Examples of mutable data types are list, set, and dictionary. These types allow adding, removing, or updating elements.
 5. Mutability helps in understanding memory usage in Python. It also helps prevent accidental changes in some data types.
-

Define Variables in Python with Example

1. A variable is a container used to store data values in a program. It helps us store and use data whenever needed.
2. In Python, we do not need to declare the data type of a variable. The variable is created automatically when we assign a value.
3. Python is a dynamically typed language. This means the type of variable is decided at runtime based on the assigned value.
4. A variable name must start with a letter or underscore. It cannot start with a number and cannot use keywords.

5. Example:

```
x = 10
```

```
name = "John"
```

Here, x stores an integer value and name stores a string value.

Explain Any Two Numeric Data Types in Python

1. Integer (int)

1. Integer data type is used to store whole numbers. These numbers do not have decimal points.
2. Examples of integers are 10, -5, and 100. Integers can be positive or negative.
3. In Python, integers can be written in decimal, binary, octal, and hexadecimal forms. For example, 0b1010 is a binary number.
4. Integer is an immutable data type. Once the value is created, it cannot be changed.

2. Float

1. Float data type is used to store decimal or floating point numbers. These numbers contain a decimal point.
2. Examples of float values are 10.5, 3.14, and -2.7. Float values can also be negative.

3. Float numbers can also be written in exponential form. For example, 1.2e3 represents 1200.0.
 4. Float is also an immutable data type. Its value cannot be changed after creation.
-

What is a String? Write Two String Operations

1. A string is a sequence of characters enclosed in single or double quotes. It is used to represent text data in Python.
 2. Examples of strings are "Python", 'Hello', and "123". Strings are immutable data types in Python.
 3. **String Operation – Indexing**
Indexing is used to access individual characters in a string. For example, s[0] gives the first character of the string.
 4. **String Operation – Slicing**
Slicing is used to get a part of a string. For example, s[1:4] returns characters from index 1 to 3.
-

Differentiate Between List and Tuple

1. A list is a collection of elements written inside square brackets. A tuple is a collection of elements written inside round brackets.
2. List is a mutable data type. Tuple is an immutable data type.
3. In a list, we can add, remove, or modify elements. In a tuple, we cannot change the elements after creation.
4. Lists are generally used when data needs to be changed frequently. Tuples are used when data should remain constant.
5. Example of list: [10, 20, 30]. Example of tuple: (10, 20, 30).

What are the Different Types of Python Distributions

1. CPython

CPython is the standard and most widely used distribution of Python. It is maintained by the Python Software Foundation.

2. Jython (JPython)

Jython is a distribution of Python that runs on the Java platform. It works on the Java Virtual Machine (JVM).

3. IronPython

IronPython is designed to run on the .NET platform. It is mainly used with C# and other .NET languages.

4. PyPy

PyPy is a Python distribution that provides better performance. It uses a JIT (Just-In-Time) compiler to increase execution speed.

5. Anaconda Python

Anaconda is mainly used for data science and large data processing. It comes with many scientific and data analysis libraries pre-installed.

Explain the range() Function with Syntax and Usage in for Loop

1. The range() function is used to generate a sequence of numbers. It is commonly used in loops.
2. The syntax of range() is:
range(stop)
range(start, stop)
range(start, stop, step)
3. The stop value is not included in the output. The sequence starts from 0 by default if start is not given.
4. The step value decides the increment between numbers. If step is not given, it increases by 1.
5. It is commonly used in for loops to repeat statements multiple times.
Example:

```
for i in range(1, 6):  
    print(i)
```

Compare and Contrast Python with Java or C++

1. Python is a dynamically typed language. Java and C++ are statically typed languages.
2. Python has simple and easy syntax. Java and C++ have more complex syntax compared to Python.
3. Python is an interpreted language. Java and C++ are compiled languages.
4. In Python, we do not need to declare variable types. In Java and C++, we must declare variable types before use.
5. Python supports rapid development and requires fewer lines of code. Java and C++ usually require more code to perform the same task.

(5 marks)

1) Explain the Steps Involved in Setting Up the Python Environment

1. First, we need to download Python from the official website python.org. We should download the latest stable version suitable for our operating system.
2. After downloading, run the installer file. During installation, we must select the option “Add Python to PATH”.
3. Once installation is complete, we can verify it using Command Prompt. We type `python --version` to check if Python is installed properly.
4. We can write Python programs using simple editors like Notepad or advanced IDEs like PyCharm. IDEs provide features like code suggestions and debugging.
5. To run a Python program, save the file with a `.py` extension. Then open Command Prompt and type `python filename.py` to execute the program.

2) Describe Python Data Types with Suitable Examples

1. Data types are used to define the type of data stored in a variable. Python supports different built-in data types.
2. Numeric data types include int, float, and complex. For example, a = 10 is int and b = 3.14 is float.
3. Boolean data type stores logical values True or False. For example, flag = True.
4. String data type is used to store text values. For example, name = "John" stores a sequence of characters.
5. Collection data types include list, tuple, set, and dictionary. For example, l = [10, 20, 30] is a list and t = (1, 2, 3) is a tuple.

3) Explain Decision-Making Statements in Python with Examples

1. Decision-making statements are used to control the flow of a program. They execute code based on certain conditions.
2. The if statement is used to check a condition. If the condition is true, the block of code inside it is executed.
3. The if-else statement is used when there are two possible outcomes. One block runs if the condition is true and another runs if it is false.
4. The elif statement is used to check multiple conditions. It allows us to test more than one condition in a program.
5. Example:

```
a = 10
if a > 5:
    print("Greater")
else:
    print("Smaller")
```

4) Write a Python Program to Check Whether a Number is Even or Odd

1. To check whether a number is even or odd, we use the modulus operator %. If a number is divisible by 2, then it is even.
2. If the remainder after dividing by 2 is not zero, then the number is odd. We use an if-else statement to check this condition.
3. The program first takes input from the user. Then it checks the condition using `num % 2 == 0`.
4. If the condition is true, it prints "Even number". Otherwise, it prints "Odd number".
5. Example Program:

```
num = int(input("Enter a number: "))
    if num % 2 == 0:
        print("Even number")
    else:
        print("Odd number")
```

5) Explain Different Types of Loops in Python with Examples

1. Loops are used to execute a block of code repeatedly. They help reduce repetition in programs.
2. **For Loop**
The for loop is used when we know how many times we want to repeat. It is commonly used with the `range()` function.
Example:

```
for i in range(1, 6):
    print(i)
```

3. While Loop

The while loop is used when we want to repeat a block until a condition becomes false. It checks the condition before executing the loop body.

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

4. Nested Loop

A loop inside another loop is called a nested loop. It is used when we need repeated operations inside another repetition.

5. Loops help in tasks like printing numbers, processing lists, and creating patterns. They are very important in programming.

6) Describe a List and Dictionary with Suitable Examples

1. List

A list is an ordered collection of elements written inside square brackets. Lists can store different types of data like numbers and strings.

2. Lists are mutable, which means we can change their elements. We can add, remove, or modify items in a list.

3. Example of list:

```
numbers = [10, 20, 30, 40]
print(numbers[0])
```

4. Dictionary

A dictionary is a collection of key-value pairs written inside curly brackets. Each key is unique and used to access its value.

5. Dictionaries are also mutable. Example:

```
student = {"name": "John", "age": 20}
print(student["name"])
```


7) Explain String Slicing with Examples

1. String slicing means getting a part of a string using index values. It is done using the slice operator [:].
2. The general syntax of slicing is string[start : end]. The start index is included, but the end index is not included.
3. Strings follow zero-based indexing in Python. The first character has index 0.
4. Slicing can also use negative indexing. Negative index starts from the end of the string.
5. Example:

```
s = "PythonProgramming"
print(s[0:6]) # Output: Python
print(s[10:14]) # Output: gram
```

Here, "Python" is extracted using s[0:6] and "gram" is extracted using s[10:14].

8) Differentiate Between list.append(), list.extend(), and list.insert()

1. append() is used to add one element at the end of the list. It adds the element as a single item.

```
l = [1, 2, 3]
l.append(4)
print(l) # [1, 2, 3, 4]
```

2. extend() is used to add multiple elements from another collection. It adds each element separately to the list.

```
l = [1, 2, 3]
l.extend([4, 5])
print(l) # [1, 2, 3, 4, 5]
```

3. insert() is used to add an element at a specific position. It takes index and value as arguments.

```
l = [1, 2, 3]
l.insert(1, 10)
print(l) # [1, 10, 2, 3]
```

4. `append()` always adds at the end of the list. `insert()` can add at any position.
5. `extend()` is mainly used when adding multiple elements. `append()` and `insert()` usually add one element at a time.

9) Describe Membership and Identity Operators in Python

1. Membership operators are used to check whether a value exists in a collection. The two membership operators are `in` and `not in`.
2. Example of membership operator:

```
x = "Python"
print("P" in x)    # True
print("z" not in x) # True
```

Here, `in` checks if a value is present inside the string.

3. Identity operators are used to compare whether two variables refer to the same object. The two identity operators are `is` and `is not`.
4. Example of identity operator:

```
a = 10
b = 10
print(a is b) # True
```

Here, `is` checks whether both variables point to the same memory location.

5. The `in` operator checks for value presence in a collection. The `is` operator checks whether two variables refer to the same object in memory.

10) How do Dictionaries Differ from Lists in Terms of Storage and Access?

1. A list stores elements in an ordered sequence. A dictionary stores data in key-value pairs.
2. In a list, elements are accessed using index numbers. In a dictionary, values are accessed using keys.
3. Lists allow duplicate elements. Dictionary keys must be unique.
4. Lists are written inside square brackets []. Dictionaries are written inside curly brackets { }.
5. Lists are useful when order matters. Dictionaries are useful when we want to store related data with a unique key.

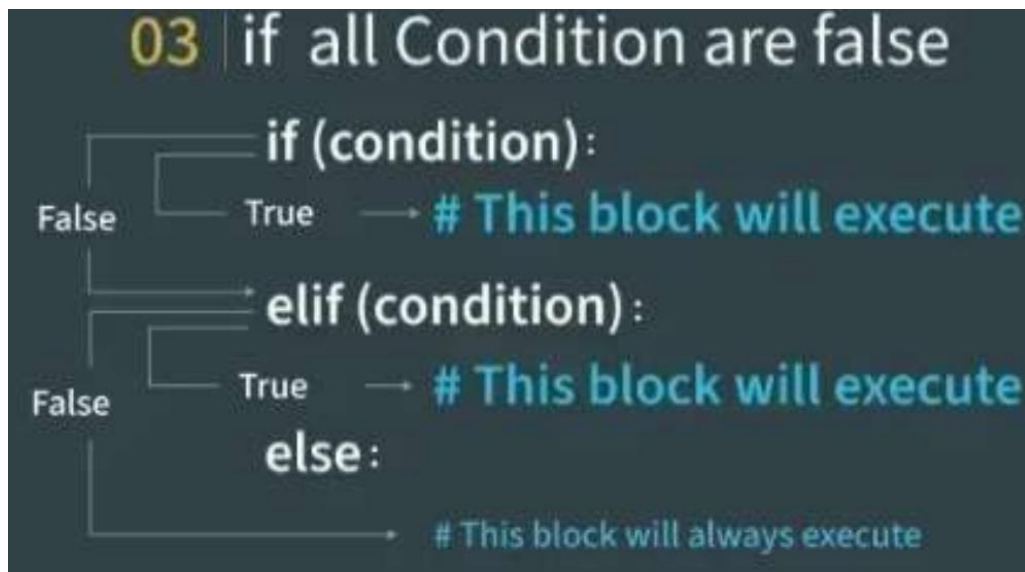
Example: Dictionary of Three Students and Their Marks

```
students = {  
    "Rahul": 85,  
    "Sneha": 90,  
    "Amit": 78  
}
```

```
print(students["Rahul"])
```

11) Explain the if-elif-else Ladder with a Flowchart

1. The if-elif-else ladder is used to check multiple conditions. It executes the block of code for the first true condition.
2. The if statement checks the first condition. If it is false, the program checks the elif condition.
3. We can use multiple elif statements. If none of the conditions are true, the else block is executed.
4. The flowchart structure is:



5.

6. This structure helps in decision-making when there are many choices. It improves program control and readability.

Program to Find Largest of Three Numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
```

```
if a >= b and a >= c:
    print("Largest number is:", a)
elif b >= a and b >= c:
    print("Largest number is:", b)
else:
    print("Largest number is:", c)
```

12) What is a Tuple? Why Use Tuple Instead of List?

1. A tuple is an ordered collection of elements written inside round brackets (). It can store different types of data.
2. Tuples are immutable, which means we cannot change their elements after creation. We cannot add, remove, or modify items.
3. Lists are mutable, but tuples are not. This makes tuples more secure for fixed data.
4. We use tuples when data should not change, such as coordinates. For example, (10, 20) represents x and y positions.
5. Tuples use less memory compared to lists. They are also faster when accessing fixed data.

Example:

```
coordinates = (10, 20)  
print(coordinates[0])
```

(10 marks)

Q. Explain the Features and Advantages of Python. Compare Python with Other Programming Languages.

Python is a simple and easy-to-learn programming language. Its syntax is clear and readable, which makes it suitable for beginners.

Python is a high-level programming language. Programmers do not need to handle low-level details like memory management.

Python is platform independent. A Python program written on Windows can run on Linux or Mac without modification.

Python is dynamically typed. We do not need to declare the data type of a variable before using it.

Python has a large standard library. It provides built-in modules for web development, data analysis, automation, and more.

Compared to Java and C++, Python has shorter and simpler code. Java and C++ require more lines of code for the same task.

Python is an interpreted language, while C++ is compiled. This makes Python slower but easier to debug.

Java and C++ are statically typed languages. Python is dynamically typed, which provides flexibility and faster development.

Q. Explain Variables, Data Types, and Type Conversion in Python with Examples.

A variable is a container used to store data values. It is created automatically when we assign a value to it.

Example:

```
x = 10
name = "John"
```

Data types define the kind of value a variable holds. Python supports int, float, complex, bool, and str as basic data types.

Example:

```
a = 5    # int
b = 3.14 # float
c = True # bool
```

Python also supports collection data types like list, tuple, set, and dictionary. For example, `l = [1,2,3]` is a list.

Type conversion means converting one data type into another. Python provides functions like `int()`, `float()`, `str()`, and `bool()` for conversion.

Example:

```
x = "10"
y = int(x)
print(y + 5)
```

Here, the string "10" is converted into an integer. Type conversion helps perform mathematical and logical operations correctly.

Q. Write a Python Program Using Loops to Print the Multiplication Table of a Number

1. A multiplication table shows the result of multiplying a number with other numbers. We can use a loop to repeat the multiplication process.
2. The for loop is commonly used when we know how many times to repeat. We usually print the table from 1 to 10.
3. First, we take input from the user. Then we use the range() function inside the loop.
4. Inside the loop, we multiply the given number with the loop variable. The result is printed in a proper format.
5. Example Program:

```
num = int(input("Enter a number: "))
```

```
    for i in range(1, 11):  
        print(num, "x", i, "=", num * i)
```

6. This program runs the loop 10 times. Each time it prints one line of the multiplication table.

Q. Write a Python Program to Count Vowels and Consonants in a Given String

1. Vowels in English are a, e, i, o, u. All other alphabets are called consonants.
2. First, we take a string input from the user. Then we convert it to lowercase for easy checking.
3. We create two variables to count vowels and consonants. These variables are increased inside the loop.
4. We use a for loop to check each character in the string. If the character is a vowel, we increase the vowel count.
5. If the character is an alphabet but not a vowel, it is counted as a consonant. Spaces and special characters are ignored.
6. Example Program:

```
string = input("Enter a string: ").lower()
```

```
    vowels = 0  
    consonants = 0
```

```
    for ch in string:  
        if ch in "aeiou":  
            vowels += 1  
        elif ch.isalpha():  
            consonants += 1
```

```
    print("Vowels:", vowels)  
    print("Consonants:", consonants)
```

7. This program checks each letter one by one. Finally, it prints the total number of vowels and consonants

Q. Explain Lists, Tuples, Sets, and Dictionaries with Syntax, Characteristics, and Examples

List

1. A list is an ordered collection of elements written inside square brackets []. It can store different types of data like numbers and strings.
2. Lists are mutable, which means we can change their elements after creation. We can add, remove, or modify items.
3. Lists allow duplicate values. They also maintain insertion order.
4. Syntax:

```
list_name = [value1, value2, value3]
```

5. Example:

```
numbers = [10, 20, 30]
numbers.append(40)
print(numbers)
```

Tuple

1. A tuple is an ordered collection written inside round brackets (). It is similar to a list but has one major difference.
2. Tuples are immutable, which means we cannot change their elements. Once created, values cannot be modified.
3. Tuples allow duplicate elements. They maintain the order of insertion.
4. Syntax:

```
tuple_name = (value1, value2, value3)
```

5. Example:

```
t = (1, 2, 3)
print(t[0])
```

Set

1. A set is an unordered collection written inside curly brackets { }. It stores only unique elements.
2. Sets do not allow duplicate values. If duplicates are given, they are automatically removed.
3. Sets are mutable, but their elements must be immutable. They do not maintain order.
4. Syntax:

```
set_name = {value1, value2, value3}
```

5. Example:

```
s = {1, 2, 3, 2}  
print(s)
```

Dictionary

1. A dictionary is a collection of key-value pairs written inside curly brackets { }. Each key is unique.
2. Dictionaries are mutable. We can add, update, or delete key-value pairs.
3. Values can be duplicated, but keys must be unique. Data is accessed using keys, not index.
4. Syntax:

```
dict_name = {key1: value1, key2: value2}
```

5. Example:

```
student = {"name": "John", "age": 20}  
print(student["name"])
```

Q. Write a Python Program to Perform Basic Operations on a List (Insert, Delete, Search, and Sort)

1. A list supports many built-in functions for operations. We can insert, delete, search, and sort elements easily.
2. Insert operation can be done using `append()` or `insert()`. Delete operation can be done using `remove()` or `pop()`.
3. Search operation can be done using the `in` operator. Sorting can be done using the `sort()` method.
4. Example Program:

```
nums = [10, 30, 20, 40]
```

```
# Insert
```

```
nums.append(50)
```

```
nums.insert(1, 15)
```

```
# Delete
```

```
nums.remove(30)
```

```
# Search
```

```
if 20 in nums:
```

```
    print("20 found in list")
```

```
# Sort
```

```
nums.sort()
```

```
print("Final list:", nums)
```

5. This program shows basic list operations clearly. Lists are very flexible and useful in Python programming.

Q . Practical Logic Implementation: Menu-Driven Program for List Operations

```
my_list = []

while True:
    print("\n----- MENU -----")
    print("1. Add an element")
    print("2. Remove an element by value")
    print("3. Reverse the list")
    print("4. Sort the list in descending order")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        element = int(input("Enter element to add: "))
        my_list.append(element)
        print("Updated List:", my_list)

    elif choice == "2":
        element = int(input("Enter element to remove: "))
        if element in my_list:
            my_list.remove(element)
            print("Updated List:", my_list)
        else:
            print("Element not found in the list.")

    elif choice == "3":
        my_list.reverse()
        print("Reversed List:", my_list)

    elif choice == "4":
        my_list.sort(reverse=True)
        print("Sorted List (Descending):", my_list)

    elif choice == "5":
        print("Exiting program.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Q . 1. Explain How to Merge Two Dictionaries

1. Merging dictionaries means combining two dictionaries into one. The final dictionary contains key-value pairs from both dictionaries.
2. If both dictionaries have unique keys, all key-value pairs are added. If the same key exists in both, the latest value overwrites the old one.
3. We can merge dictionaries using the `update()` method. This method adds elements of one dictionary into another.
4. Example using `update()`:

```
dict1 = {"a": 1, "b": 2}  
dict2 = {"c": 3, "d": 4}
```

```
dict1.update(dict2)  
print(dict1)
```

5. We can also merge dictionaries using the `|` operator in Python 3.9 and above. This creates a new merged dictionary.

```
d1 = {"x": 10, "y": 20}  
d2 = {"z": 30}
```

```
merged = d1 | d2  
print(merged)
```

6. Dictionary merging is useful when combining related data. It helps manage data efficiently in large programs.

II. Discuss Set Operations: Union, Intersection, Difference, and Symmetric Difference

1. Sets are unordered collections of unique elements. They support mathematical operations like union and intersection.

1. Union (|)

2. Union combines elements from both sets without duplicates. It returns all unique elements from both sets.

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
print(set1 | set2)
```

2. Intersection (&)

3. Intersection returns common elements present in both sets. Only shared elements are included in the result.

```
print(set1 & set2)
```

3. Difference (-)

4. Difference returns elements that are in the first set but not in the second set. It removes common elements.

```
print(set1 - set2)
```

4. Symmetric Difference (^)

5. Symmetric difference returns elements that are in either set but not in both. It removes common elements and keeps unique ones.

```
print(set1 ^ set2)
```

6. These set operations are useful in comparing data. They help in solving problems related to unique and common elements.

Q . Built-in String Methods in Python

1. **split()**

The `split()` method divides a string into a list of words. By default, it splits using spaces.

Example:

```
s = "Python is easy"
print(s.split())
```

2. **join()**

The `join()` method joins elements of a list into a single string. It uses a specified separator.

Example:

```
words = ["Python", "is", "easy"]
print(" ".join(words))
```

3. **strip()**

The `strip()` method removes extra spaces from the beginning and end of a string. It is useful for cleaning user input.

Example:

```
s = " Hello "
print(s.strip())
```

4. **find()**

The `find()` method searches for a substring in a string. It returns the index position if found, otherwise -1.

Example:

```
s = "Python"
print(s.find("th"))
```

5. **replace()**

The `replace()` method replaces one substring with another. It returns a new modified string.

Example:

```
s = "I like Java"
print(s.replace("Java", "Python"))
```

6. These string methods help in processing and modifying text data. They are very useful in real-world applications like form validation and text analysis.

Python Script to Count Number of Vowels in a Sentence

1. First, we take a sentence as input from the user. Then we convert it to lowercase for easy checking.
2. We define vowels as a, e, i, o, u. We create a counter variable to count vowels.
3. We use a for loop to check each character in the sentence. If the character is a vowel, we increase the count.
4. Finally, we print the total number of vowels. This program ignores spaces and special characters automatically.

Program:

```
sentence = input("Enter a sentence: ").lower()

vowel_count = 0

for ch in sentence:
    if ch in "aeiou":
        vowel_count += 1

print("Number of vowels:", vowel_count)
```

5. This script checks each letter one by one. It gives the total number of vowels present in the sentence.

Q. Comparison of List, Tuple, Set, and Dictionary

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{key:value}
Ordering	Ordered	Ordered	Unordered	Ordered (Python 3.7+)
Mutability	Mutable	Immutable	Mutable	Mutable
Duplicates Allowed	Yes	Yes	No	Keys: No, Values: Yes

Explanation in Points

1. Syntax

Lists use square brackets, tuples use round brackets. Sets and dictionaries use curly brackets, but dictionaries store key-value pairs.

2. Ordering

Lists and tuples maintain insertion order. Sets do not maintain order, while dictionaries maintain insertion order in modern Python versions.

3. Mutability

Lists, sets, and dictionaries are mutable, so their data can be changed. Tuples are immutable, so their data cannot be modified after creation.

4. Duplicate Elements

Lists and tuples allow duplicate values. Sets do not allow duplicates, and dictionary keys must be unique.

Code Examples : -

1) List Example

```
my_list = [10, 20, 30, 20]
my_list.append(40)
print(my_list)
```

2) Tuple Example

```
my_tuple = (1, 2, 3, 2)
print(my_tuple)
print(my_tuple[0])
```

3) Set Example

```
my_set = {1, 2, 3, 2}
my_set.add(4)
print(my_set)
```

4) Dictionary Example

```
my_dict = {"name": "John", "age": 20}
my_dict["city"] = "Mumbai"
print(my_dict)
```

Q. I. Difference Between while Loop and for Loop

1. A **for loop** is used when we know how many times we want to repeat a block of code. A **while loop** is used when we repeat a block until a condition becomes false.
2. The for loop is commonly used with sequences like lists, strings, or range(). The while loop works based on a condition.
3. In a for loop, the loop variable changes automatically. In a while loop, we must manually update the loop variable.
4. If we forget to update the condition in a while loop, it may cause an infinite loop. The for loop is safer when the number of iterations is known.
5. Example of for loop:

```
for i in range(1, 6):  
    print(i)
```

6. Example of while loop:

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

II. Role of break, continue, and pass Statements

1. The **break** statement is used to stop the loop completely. When break is executed, the loop ends immediately.
2. Example of break:

```
for i in range(1, 6):  
    if i == 3:  
        break  
    print(i)
```

3. The **continue** statement skips the current iteration. It moves control to the next iteration of the loop.
4. Example of continue:

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

5. The **pass** statement does nothing. It is used as a placeholder when we do not want to write code yet.

6. Example of pass:

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        pass
```

```
    print(i)
```

III. Program to Print All Prime Numbers Between 1 and 50

1. A prime number is a number greater than 1 that has only two factors. These factors are 1 and the number itself.
2. We use a loop to check numbers from 2 to 50. For each number, we check if it is divisible by any other number.
3. If the number is divisible by any number other than 1 and itself, it is not prime. Otherwise, it is prime.
4. We use nested loops for checking divisibility. If no divisor is found, we print the number.

5. Program:

```
for num in range(2, 51):
```

```
    is_prime = True
```

```
    for i in range(2, num):
```

```
        if num % i == 0:
```

```
            is_prime = False
```

```
            break
```

```
    if is_prime:
```

```
        print(num)
```

6. This program checks each number one by one. It prints all prime numbers between 1 and 50.

Q. (a) Compare Python Lists and Dictionaries (4 Marks)

1. Internal Structure

A list stores elements in a sequential order. A dictionary stores data in key-value pairs using a hash table internally.

2. Indexing Method

List elements are accessed using numeric index positions starting from 0. Dictionary elements are accessed using unique keys instead of index numbers.

3. Ordering

Lists maintain insertion order of elements. Dictionaries also maintain insertion order in Python 3.7 and above.

4. Performance

Accessing an element in a list takes more time if searching is required. Accessing a value in a dictionary using a key is faster because of hashing.

5. Use Case

Lists are used when order and duplication are important.

Dictionaries are used when data needs to be stored and accessed using unique keys.

(b) Explain Dictionary Comprehension (3 Marks)

1. Dictionary comprehension is a short and simple way to create dictionaries. It is similar to list comprehension but creates key-value pairs.

2. It uses curly brackets {} with a loop and optional condition. This makes the code shorter and more readable.

3. Example:

```
squares = {x: x*x for x in range(1, 6)}  
print(squares)
```

4. In this example, keys are numbers and values are their squares. The output will be {1:1, 2:4, 3:9, 4:16, 5:25}.

(c) Program to Convert Two Lists into a Dictionary (3 Marks)

1. We can use the zip() function to combine two lists. The first list will act as keys and the second list will act as values.
2. The dict() function converts the zipped result into a dictionary. Both lists should have the same length.
3. Program:

```
names = ["Rahul", "Sneha", "Amit"]
```

```
marks = [85, 90, 78]
```

```
student_dict = dict(zip(names, marks))
```

```
print(student_dict)
```

4. This program combines names and marks into key-value pairs. The output will be a dictionary storing student names with their marks.